NETRAX - Smooth video with all cameras

Why the video hangs, what we changed, how to test it, what to do next

Models, weights, detection logic and earlier work unchanged - every change has an on/off switch + a backup

1. What the logs on the laptop show (measured, not guessed)

Situation (data/edge_run.log + docs/eval/latency_report.md)	AI frames per second	What it means
1 camera (laptop webcam), nobody in view	11.9 fps; GPU batch 57 ms	the AI models themselves are fast enough
2 cameras (laptop + phone), people in view, first minute	about 3 fps each	something besides the GPU is slow
same, after a few minutes	0.7 - 1.7 fps each	gets worse while people are in view
live picture on the dashboard	= the AI rate (1-3 fps)	this is the 'hanging' video you see
load manager in the log	'GPU busy: slowed to 4 fps'	it slowed cameras although the GPU was mostly idle

Root cause: the dashboard picture of every camera was produced by that camera's AI worker, so the video could never be smoother than the AI. The AI itself slowed down because of CPU work, not the YOLO models: face detection ran on the FULL 1920-pixel phone frame for every camera behind one shared lock, and the phones send 1080p MJPEG at 30 fps that the laptop must decode. On top of that the load manager reduced the frame rate further. The YOLO models (ppe_v2, ppe_v3B, pose, fire_v2B, shoe classifier) are **not** the problem.

2. The ChatGPT plan, checked against our code

ChatGPT suggestion	Our decision	Why
Separate capture threads, newest frame only, one central YOLO worker	Already in NETRAX	edge/stream.py (one reader per camera, newest frame) + edge/engine.py (one shared batched engine)
Show raw video separately from AI video	DONE (better variant)	new edge/live.py: smooth newest frame WITH the latest AI boxes drawn on it - one view, never frozen
Use FP16 (half precision)	REJECTED	GTX 1650 / 16xx cards give wrong results in FP16; project rule = FP32
Switch to YOLOv8n / smaller model / retrain	REJECTED	GPU is not the bottleneck (57 ms); a smaller model only loses PPE accuracy
Build a new separate camera_engine.py + server.py	REJECTED	would duplicate and break the alert, entrance, clip, face-ID logic we already have
Lower phone resolution / fps, JPEG quality 60-75	ACCEPTED (tool + steps)	model looks at 640 px anyway: 960x540 loses no accuracy, about 4x less traffic + decoding
Dedicated 5 GHz Wi-Fi, laptop on cable	ACCEPTED (setup advice)	today one phone is BOTH camera and hotspot - it overheats and stalls
Test cameras without AI first	DONE (tool)	python -m tools.camdiag tells camera/network problems from processing problems
stream_buffer=False / .streams file	NOT APPLICABLE	only for Ultralytics' own video loader; we read cameras ourselves
RTSP / H.264 instead of MJPEG	LATER, optional	less bandwidth but adds decoder delay; try only if camdiag still shows phone freezes

3. What is already changed on the laptop (installed 4 Nov)

Change	Module	Effect	Switch / backup
Smooth live view	edge/live.py (new), worker.py, run_edge.py	Every camera is shown at 12 frames/s from its NEWEST frame, with the latest AI boxes and PPE lines drawn on it. The AI can run at any speed - the video no longer freezes. A corner label shows the real AI rate ('AI 6.2 fps'). Incident clips are now made from this smooth picture.	processing.live_view: false = old way
Face detection on a smaller copy	edge/identity.py	Face DETECTION runs on a copy of at most 960 px (phones send 1920 px) = about 4x less CPU; boxes are mapped back exactly. Face RECOGNITION still uses the full-size picture, so names stay as accurate as before.	identity.detect_max_side (0 = off)
Profiler	worker.py, run_edge.py	Every 10 s the edge window prints [perf] lines: time per stage per camera (AI engine, tracker, face ID, PPE matching, decisions + fire, drawing), GPU busy share, face-detect time. We can now SEE the bottleneck.	always on (cheap)
Load manager fixed	engine.py, load.py	A camera is slowed only when the GPU is really busy (>= 85 % of the time). Before, it slowed cameras when the CPU was the problem, which only made alerts later.	load.gpu_busy_min
Missing USB camera	stream.py	A USB camera that is not plugged in is retried every 30 s instead of every 10 s (Windows camera probing briefly disturbs the other webcam).	-
Camera-only test	tools/camdiag.py (new)	All cameras together, no AI: fps, longest freeze, verdict per camera.	-
Phone settings tool	tools/phone_tune.py (new)	Shows and sets IP Webcam resolution + JPEG quality over the network (only reports a change after reading it back from the phone).	-

Not touched: models/*.pt, thresholds, PPE association, state machine (alert timing), entrance logic, backend, dashboards, database. Tests: edge.selftest, access_selftest, hw_selftest pass; final_selftest passes plus 4 new [LIVE] tests (live view 12 fps while AI at 1 fps, clips fed, profiler, face scaling). Old copies of all changed files: **_to_delete\before_live_view_4nov**.

4. Test it now (20 minutes, in this order)

Step 1 - phones (biggest network win). Open IP Webcam on each phone, Video preferences: **Video resolution 960x540** (or 800x600 / 1280x720 if 960x540 is not offered), **Quality 55**, **FPS limit 15**. Keep the phones on chargers. Or let the tool do the first two:

```
cd C:\Users\Satyabrata\Desktop\sentineleye
.\venv\Scripts\activate
python -m tools.find_phones --write      (if the phone addresses changed)
python -m tools.phone_tune               (shows current settings)
python -m tools.phone_tune --apply
```

Step 2 - cameras alone, no AI (close the NETRAX windows first):

```
python -m tools.camdiag --seconds 30
```

Every camera should say **smooth**. If a phone says **FREEZES without AI**, the problem is the phone or the network (see section 5) - no code change can fix that. Result saved in docs\eval\camdiag.md.
Step 3 - full system:

```
.\run_all.bat
(walk in front of all cameras for 2-3 minutes, take a helmet off once)
python -m tools.latency_report --label "all cameras after live view" --seconds 120
```

Expected: dashboard video smooth on every tile (about 12 pictures/s), boxes follow people with a short delay, corner label shows the AI rate. Copy the **[perf]** lines from the 'NETRAX edge' window - they show which stage still costs the most and decide step 6.

5. Network setup (removes the phone 'hang')

Today	Better	Why
Phone 1 is the Wi-Fi hotspot AND a camera	Laptop makes the hotspot (Windows Settings > Mobile hotspot, 5 GHz band if offered) or use any small router; both phones only stream	a phone that streams video and runs the hotspot heats up, throttles and drops frames - that is the 'mobile hang'
1080p, quality 80-90, 30 fps MJPEG per phone	960x540, quality 55, 15 fps	about 4x less data; the AI uses 640 px anyway
Phones far from the hotspot	within a few metres, no walls	MJPEG has no buffering - weak Wi-Fi shows as freezes at once
Other phones / laptops on the same hotspot	only the cameras, the laptop and the ESP32	every extra device shares the same air time

After changing the hotspot the phone IP addresses change: run python -m tools.find_phones --write. The ESP32 sketch must point to the laptop's new IP (re-flash) - see CLAUDE.md, 2 Nov section.

6. Next steps - only if the [perf] lines still show a bottleneck

Decide from the measured numbers, one change at a time, re-run tools.latency_report after each. Prompts below are ready for Claude Code in VS Code (open the sentineleye folder first).

[perf] shows	Do this	Accuracy risk
face_id large (> 40 ms p50)	identity.every_n_frames 3 -> 5 once a person has a name; detect_max_side 960 -> 720	none / tiny (names may take ~0.3 s longer)
ai_engine large and GPU busy > 0.85	python -m tools.export_engines + models.runtime: tensorrt (FP32, kept only if faster)	none (same weights, FP32)
cam fps low in camdiag	phone settings / hotspot (section 5); USB: other port, more light	none
decisions_fire large	fire_every_n 2 -> 3 while no candidate (already every frame when a flame is seen)	small, fire still every frame on candidates
draw_emit large	processing.live_width 800 -> 640	none (display only)
everything fine	record multi-camera numbers for the jury (latency_report) and stop changing	-

Prompt A (read-only check after a test run):

```
Read CLAUDE.md (4 Nov section) and the last [perf] and [edge] lines in data/edge_run.log.
Do NOT change code. Tell me per camera: AI fps, live fps, the slowest stage, GPU busy share,
and which ONE row of section 6 of docs/NETRAX_Smooth_Video_Plan.pdf applies.
```

Prompt B (apply one tuning step):

```
Apply only this step: <paste the row>. Change settings.yaml only (no model files, no thresholds of
PPE/fire decisions). Back up the file to _to_delete/ first. Run python -m edge.selftest and
python -m tools.final_selftest. Then tell me what to measure with tools.latency_report.
```

Prompt C (rollback, if anything looks worse):

```
Set processing.live_view: false and identity.detect_max_side: 0 in config/settings.yaml, or copy the files
back from _to_delete/before_live_view_4nov/ to the same folders. Then run the selftests.
```

7. What to say to the jury

"With several cameras the live video used to wait for the AI. We measured it: the GPU needed only 57 ms per batch, the time went to CPU work. We separated the live view from the AI - video now runs at 12 frames per second on every camera while the AI result is drawn on top - reduced face-detection cost 4x without touching our trained models, and tuned the phone streams. Here are the before/after numbers." (use docs\eval\latency_report.md and camdiag.md - measured, not estimated)